A PROJECT REPORT
ON
DATA VISUALIZATION WITH PYTHON

UNITED UNIVERSITY

FACULTY OF COMPUTER APPLICATIONS

SESSION 2025-26

SUBMITTED BY-

PURNIMA SHUKLA

BCA-IBM SECTION-"D"

UU ID—UU252010046

SUPERVISED BY—MR. FARHAN

UNIVERSITY RAWATPUR, PRAYAGRAJ, UTTAR PRADESH – 211012

INTRODUCTION

Smart City Analytics is a transformative approach to urban management that leverages data to enhance the quality of life, sustainability, and efficiency of city operations. In an era of rapid urbanization, cities face significant challenges in managing environmental health and transportation infrastructure. By utilizing advanced data visualization techniques, complex urban datasets can be converted into actionable insights that help city planners and policymakers make informed decisions. Python serves as the core technology for this analysis due to its robust ecosystem of specialized libraries. This project utilizes Pandas and NumPy for sophisticated data handling, while Matplotlib and Seaborn are employed to create high-impact visual representations of urban trends. This project focuses on a multi-dimensional analysis using two primary datasets:

1. **Air Quality Dataset:** Tracks critical environmental pollutants including PM2.5, PM10, NOx, NO2, SO2, VOCs, CO, CO2, and CH4, alongside meteorological factors like temperature, humidity, and wind direction.

2. Traffic Prediction Dataset: Monitors urban mobility through datetime, junction identifiers, and vehicle counts. By performing Exploratory Data Analysis (EDA), this project aims to identify correlations between traffic congestion and air pollution levels across various location types. The integration of these datasets provides a holistic view of the city's pulse, forming a foundation for smarter, greener urban development.

OBJECTIVES OF THE PROJECT

The primary goal of this project is to leverage data visualization to understand urban dynamics and propose data-driven solutions for a smarter, more sustainable city. The specific objectives are:

- **To Integrate Diverse Urban Datasets:** Effectively combine environmental data (Air Quality) and mobility data (Traffic) to create a unified view of city health.
- **To Perform Rigorous Data Pre-processing:** Clean the raw datasets by handling missing pollutant values, correcting sensor errors, and formatting temporal data for accurate time-series analysis.
- **To Analyze Environmental Impact:** Identify the distribution and variance of critical pollutants like **PM2.5, NO2, and CO2** across different urban zones (Industrial vs. Residential).
- **To Identify Traffic Congestion Patterns:** Detect peak hours and "bottleneck" junctions to understand when and where the city's infrastructure is under the most pressure.
- **To Study Correlation between Mobility and Pollution:** Visualize the direct relationship between vehicle density and the rise of harmful gases to validate the need for green corridors.
- **To Support Urban Decision-Making:** Create high-impact visual dashboards (Heatmaps, Scatter plots, and Trend lines) that translate complex numbers into actionable insights for city planners.
- **To Demonstrate Technical Proficiency:** Utilize industry-standard Python libraries including **Pandas** for data manipulation and **Seaborn/Matplotlib** for advanced statistical visualization.

DATA CLEANING & PREPROCESSING

To ensure analytical accuracy, the following data cleaning steps are performed on the raw datasets:

- **Handling Missing Values:** Identification and treatment of null entries in pollutant levels (e.g., PM2.5, SO2) and traffic counts using mean/median imputation or removal to prevent skewed visualizations.
- **DateTime Conversion:** Transforming the `datetime` column in the traffic dataset into standard Python datetime objects to extract hourly, daily, and monthly patterns.
- **Data Type Standardization:** Ensuring all pollutant metrics and vehicle counts are in numeric format (float/int) for statistical computation.
- **Categorical Encoding:** Processing `location_type` and `source_label` to enable grouped analysis (e.g., comparing Industrial vs. Residential air quality).
- **Outlier Detection:** Using statistical methods to identify and address anomalies in sensor data that may represent equipment errors rather than actual environmental spikes.

TOOLS AND TECHNOLOGIES USED

This project leverages the **Python Ecosystem**, which is the industry standard for data science and analytics. The following tools were utilized to transform raw urban data into meaningful insights: [1]

1. Programming Language

- **Python (Version 3.x):** Chosen for its versatility, ease of use, and extensive support for data-centric libraries. It serves as the backbone for both data cleaning and visualization logic.

2. Integrated Development Environment (IDE)

- **Jupyter Notebook / Google Colab:** An interactive web-based environment used for writing and running code. It allows for "literate programming," where code, visualizations, and explanatory text are combined in a single document.

3. Core Data Libraries

- **Pandas:** The primary tool for data manipulation and analysis. It was used to load the CSV datasets, handle missing values in pollutant columns, and perform grouping for junction-wise traffic analysis.

- **NumPy:** Used for high-performance mathematical computations and handling multi-dimensional arrays, especially when calculating mean pollutant levels and traffic statistics. [1]

4. Data Visualization Libraries

- **Matplotlib:** A fundamental plotting library used to create static, high-quality graphs such as line charts for traffic trends and histograms for pollutant distribution.
- **Seaborn:** Built on top of Matplotlib, it was used to create more complex and aesthetically pleasing statistical visualizations, including **Heatmaps** for correlation and **Boxplots** for outlier detection in air quality.

5. Data Pre-processing Techniques

- **Data Cleaning:** Techniques like `fillna()` for handling missing sensor data and `to_datetime()` for parsing traffic timestamps.
- **Feature Engineering:** Extracting "Hour," "Day," and "Month" from the datetime column to analyze peak-hour urban behavior.

DATASET DESCRIPTION

The datasets used in this project are designed for **Smart City Analytics**, focusing on urban environmental health and traffic mobility. They include detailed information on air pollutants, meteorological conditions, and vehicle counts across various city junctions. These datasets are structured in tabular form and are suitable for **Exploratory Data Analysis (EDA)** and visualization tasks.

In this project, these datasets are utilized to perform **Data Cleaning** and **Visualization** in order to extract meaningful insights about urban pollution trends and traffic congestion. The cleaned and well-analyzed data can further be used for predictive modeling and automated city management systems.

[23]:
```
# Display Top 5 Rows of Traffic Dataset
df_traf.head()

# Display Top 5 Rows of Air Quality Dataset
df_air.head()
```

[23]:

	PM2.5	PM10	NOx	NO2	SO2	VOCs	CO	CO2	CH4	Temperature	Humidity	Wind_Direction	Location_Type	Sou
0	39.967142	57.926035	116.192213	55.230299	4.531693	75.317261	2.789606	427.674347	1.706105	31.085120	45.454749	276	Urban	
1	101.935672	150.774299	76.826826	79.051618	18.744780	145.083987	1.966569	529.739619	2.492663	33.711103	60.798212	134	Industrial	
2	70.996192	138.948796	158.731020	60.466604	14.892239	145.147338	2.626446	499.889443	2.431165	33.778698	54.875669	1	Industrial	
3	28.464728	63.643900	25.385343	15.333286	7.647429	130.022319	1.779360	388.283712	1.818563	31.565877	67.113319	251	Rural	
4	78.265276	113.977926	105.644340	59.202337	17.696806	181.713667	3.240533	464.739197	2.597225	32.229835	37.236519	326	Industrial	

Dataset 1: Air Quality Data

This dataset monitors environmental health through the following columns:

1. **PM2.5 / PM10** – Fine and coarse particulate matter concentration.
2. **NOx / NO2** – Nitrogen oxide levels, often linked to vehicle exhaust.
3. **SO2** – Sulfur dioxide levels from industrial sources.
4. **VOCs** – Volatile organic compounds.
5. **CO / CO2 / CH4** – Carbon monoxide, Carbon dioxide, and Methane levels.
6. **Temperature / Humidity** – Meteorological factors affecting pollutant dispersal.
7. **Wind Direction** – Direction of airflow across the city.
8. **Location_Type** – Categorical data (e.g., Industrial, Residential, Commercial).
9. **Source_Label** – The identified source of the pollution.

Dataset 2: Traffic Prediction Data

This dataset tracks urban mobility through the following columns:

1. **DateTime** – The timestamp of the recorded traffic data.
2. **Junction** – The specific identifier for the city intersection.
3. **Vehicles** – Total count of vehicles passing through.
4. **ID** – A unique identifier for each data record.

EDA (EXPLORATORY DATA ANALYSIS)

Exploratory Data Analysis (EDA) is a crucial step in the data analysis process that involves examining datasets to summarize their main characteristics using statistical and graphical techniques. The primary goal of EDA is to understand the structure of the data, identify patterns, detect anomalies, and uncover relationships between variables before applying any machine learning models.

In this project, EDA is performed to analyze the **Smart City Analytics** dataset and gain insights into various urban environmental and mobility attributes. By visualizing the distribution of features such as **PM2.5, CO2 levels, and vehicle counts**, it becomes easier to identify outliers, skewness, and trends present in the data.

EDA also helps in identifying correlations between different variables, which is essential for understanding how certain traffic patterns influence air quality indicators. Visualization techniques such as **histograms, scatter plots, box plots, and heatmaps** are used to represent these relationships clearly.

Overall, Exploratory Data Analysis provides a strong foundation for data cleaning, transformation, and further predictive modeling. It ensures that the dataset is well-understood and reliable before being used for advanced smart city management applications.

IMPORTING LYBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

READING DATA

```
df_traf = pd.read_csv('traffic.csv')
print(df_traf.head())
df_air = pd.read_csv('air_quality_dataset.csv')
print(df_air.head())
```

```
             DateTime  Junction  Vehicles           ID
0  2015-11-01 00:00:00         1        15  20151101001
1  2015-11-01 01:00:00         1        13  20151101011
2  2015-11-01 02:00:00         1        10  20151101021
3  2015-11-01 03:00:00         1         7  20151101031
4  2015-11-01 04:00:00         1         9  20151101041
        PM2.5        PM10         NOx        NO2        SO2        VOCs  \
0   39.967142   57.926035  116.192213  55.230299   4.531693   75.317261
1  101.935672  150.774299   76.826826  79.051618  18.744780  145.083987
2   70.996192  138.948796  158.731020  60.466604  14.892239  145.147338
3   28.464728   63.643900   25.385343  15.333286   7.647429  130.022319
4   78.265276  113.977926  105.644340  59.202337  17.696806  181.713667

         CO         CO2       CH4  Temperature   Humidity  Wind_Direction  \
0  2.789606  427.674347  1.706105    31.085120  45.454749             276
1  1.966569  529.739619  2.492663    33.711103  60.798212             134
2  2.626446  499.889443  2.431165    33.778698  54.875669               1
3  1.779360  388.283712  1.818563    31.565877  67.113319             251
4  3.240533  464.739197  2.597225    32.229835  37.236519             326

  Location_Type     Source_Label
0         Urban        Vehicular
1    Industrial       Industrial
2    Industrial       Industrial
3         Rural  Biomass Burning
4    Industrial       Industrial
```

DATASET INFORMATION

```
df_traf.info()
df_air.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48120 entries, 0 to 48119
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   DateTime  48120 non-null  object
 1   Junction  48120 non-null  int64
 2   Vehicles  48120 non-null  int64
 3   ID        48120 non-null  int64
dtypes: int64(3), object(1)
memory usage: 1.5+ MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 14 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   PM2.5           500 non-null    float64
 1   PM10            500 non-null    float64
 2   NOx             500 non-null    float64
 3   NO2             500 non-null    float64
 4   SO2             500 non-null    float64
 5   VOCs            500 non-null    float64
 6   CO              500 non-null    float64
 7   CO2             500 non-null    float64
 8   CH4             500 non-null    float64
 9   Temperature     500 non-null    float64
 10  Humidity        500 non-null    float64
 11  Wind_Direction  500 non-null    int64
 12  Location_Type   500 non-null    object
 13  Source_Label    500 non-null    object
dtypes: float64(11), int64(1), object(2)
memory usage: 54.8+ KB
```

STATISTICAL SUMMARY OF DATABASE

```
[22]: df_traf.describe()
      df_air.describe()
```

[22]:

	PM2.5	PM10	NOx	NO2	SO2	VOCs	CO	CO2	CH4	Temperature	Humidity	Wind_Direction
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000
mean	54.671620	84.703251	91.814080	42.410820	11.504816	112.193098	2.177705	439.288087	2.130439	30.448413	56.196914	180.842000
std	22.799473	30.567912	54.808414	26.343669	7.306978	39.471686	0.824762	51.220578	0.391396	2.957290	11.035563	102.323767
min	10.757597	12.932337	-0.076323	-1.942140	-1.095555	27.555617	0.366992	366.063158	1.388407	19.581349	35.077851	0.000000
25%	36.372940	61.118607	35.457263	17.448632	5.976492	81.516729	1.536915	397.399610	1.837395	28.481974	47.254489	95.750000
50%	48.038599	78.135558	95.452284	39.211583	8.681222	104.819124	2.045519	420.934319	2.066709	30.432756	55.816773	183.000000
75%	73.820566	110.270086	134.709240	63.817763	17.116673	140.588100	2.726020	490.968315	2.386267	32.614954	64.022677	267.000000
max	127.616632	171.594187	225.886935	115.916428	31.941860	231.926017	4.798935	558.924257	3.352266	39.778713	79.991414	359.000000

MISSING VALUES

```
print(df_traf.isnull().sum())

print(df_air.isnull().sum())
```

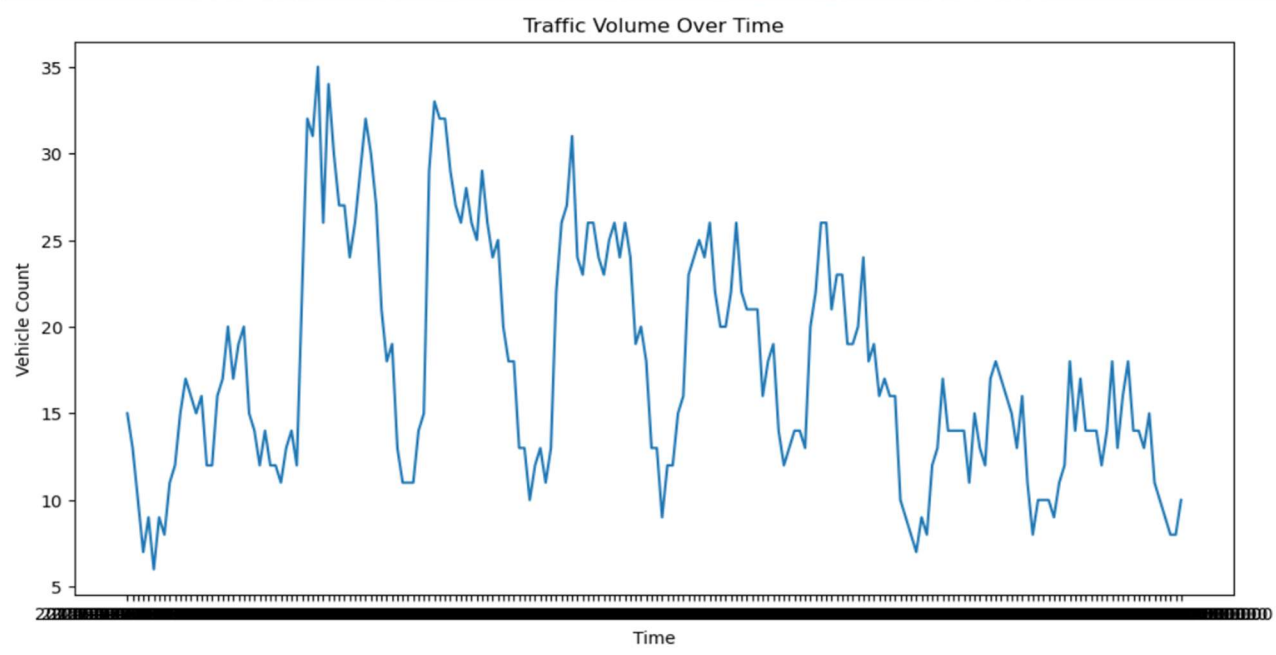
```
DateTime    0
Junction    0
Vehicles    0
ID          0
dtype: int64
PM2.5              0
PM10               0
NOx                0
NO2                0
SO2                0
VOCs               0
CO                 0
CO2                0
CH4                0
Temperature        0
Humidity           0
Wind_Direction     0
Location_Type      0
Source_Label       0
dtype: int64
```

Insight 1: Peak Hour Traffic Analysis

Objective- To identify peak traffic hours in the city.

```
plt.figure(figsize=(12,6))

plt.plot(
    df_traf['DateTime'][:200],
    df_traf['Vehicles'][:200]
)

plt.title('Traffic Volume Over Time')
plt.xlabel('Time')
plt.ylabel('Vehicle Count')

plt.show()
```

Visualization Used- Line Plot



Analysis

- Traffic volume increases significantly during peak hours.
- Certain time intervals show repeated congestion patterns.
- High vehicle counts indicate rush-hour traffic movement.

Smart City Insight

This analysis helps authorities:

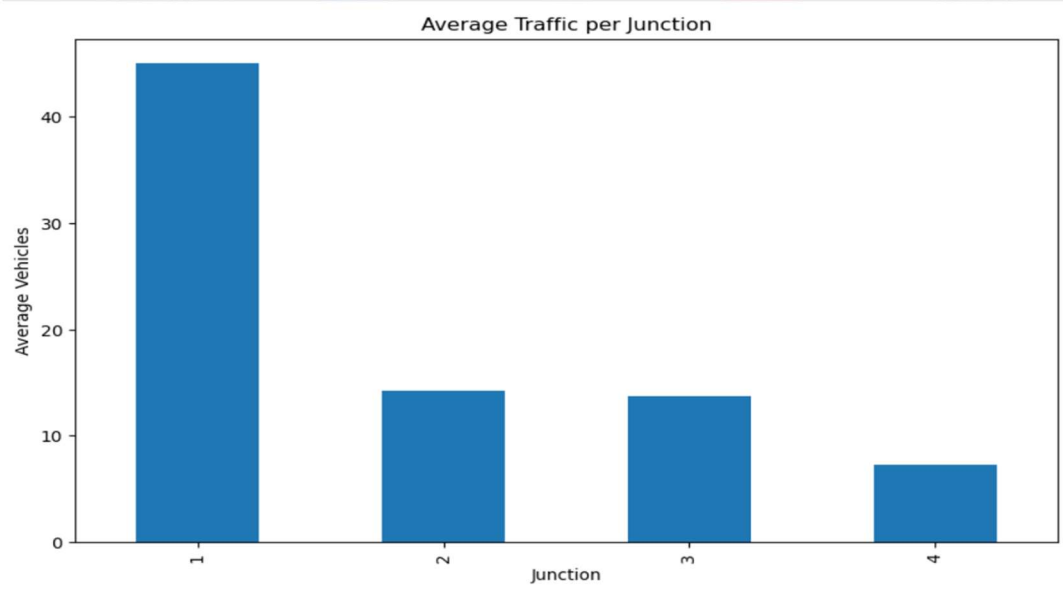
- optimize signal timing
- reduce congestion
- improve road planning
- manage peak-hour traffic efficiently

INSIGHT 2: JUNCTION-WISE TRAFFIC COMPARISON

Objective- **To compare average traffic density across city junctions.**

- Visualization Used-**Bar Plot**

```
[29]: df_traf.groupby('Junction')['Vehicles'].mean().plot(
      kind='bar',
      figsize=(10,6)
      )

      plt.title('Average Traffic per Junction')
      plt.xlabel('Junction')
      plt.ylabel('Average Vehicles')

      plt.show()
```



:

Analysis

- Some junctions consistently experience heavier traffic.
- Certain roads remain less congested.

- Traffic imbalance can be clearly observed.

Smart City Insight

This helps in:

- identifying high-density junctions
- improving traffic distribution
- planning alternate routes
- prioritizing infrastructure upgrades

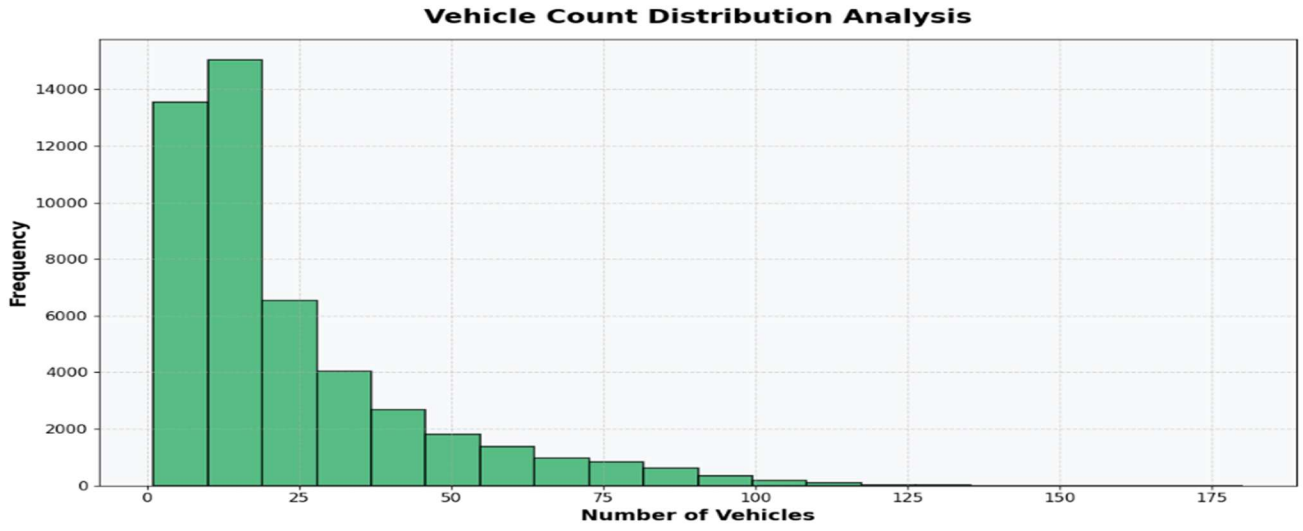
INSIGHT 3: VEHICLE DISTRIBUTION ANALYSIS

OBJECTIVE- **TO UNDERSTAND OVERALL TRAFFIC DENSITY DISTRIBUTION.**

Visualization Used-**Histogram**

Code

50]:
```
plt.figure(figsize=(12,7))

# Customized Histogram
plt.hist(
    df_traf['Vehicles'],
    bins=20,
    color='mediumseagreen',
    edgecolor='black',
    linewidth=1.2,
    alpha=0.85
)

# Title
plt.title(
    'Vehicle Count Distribution Analysis',
    fontsize=18,
    fontweight='bold',
    pad=15
)

# Axis Labels
plt.xlabel(
    'Number of Vehicles',
    fontsize=14,
    fontweight='bold'
)

plt.ylabel(
    'Frequency',
    fontsize=14,
    fontweight='bold'
)

# Grid
plt.grid(
    linestyle='--',
    alpha=0.5
)

# Background Styling
plt.gca().set_facecolor('#f8f9fa')

# Tick Styling
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

# Layout
plt.tight_layout()

plt.show()
```



Analysis

- Most traffic values fall within a moderate range.
- Extremely high vehicle counts represent congestion periods.
- Distribution helps identify traffic density patterns.

Smart City Insight

This analysis supports:

- congestion prediction
- traffic monitoring
- smart road management
- urban transportation planning

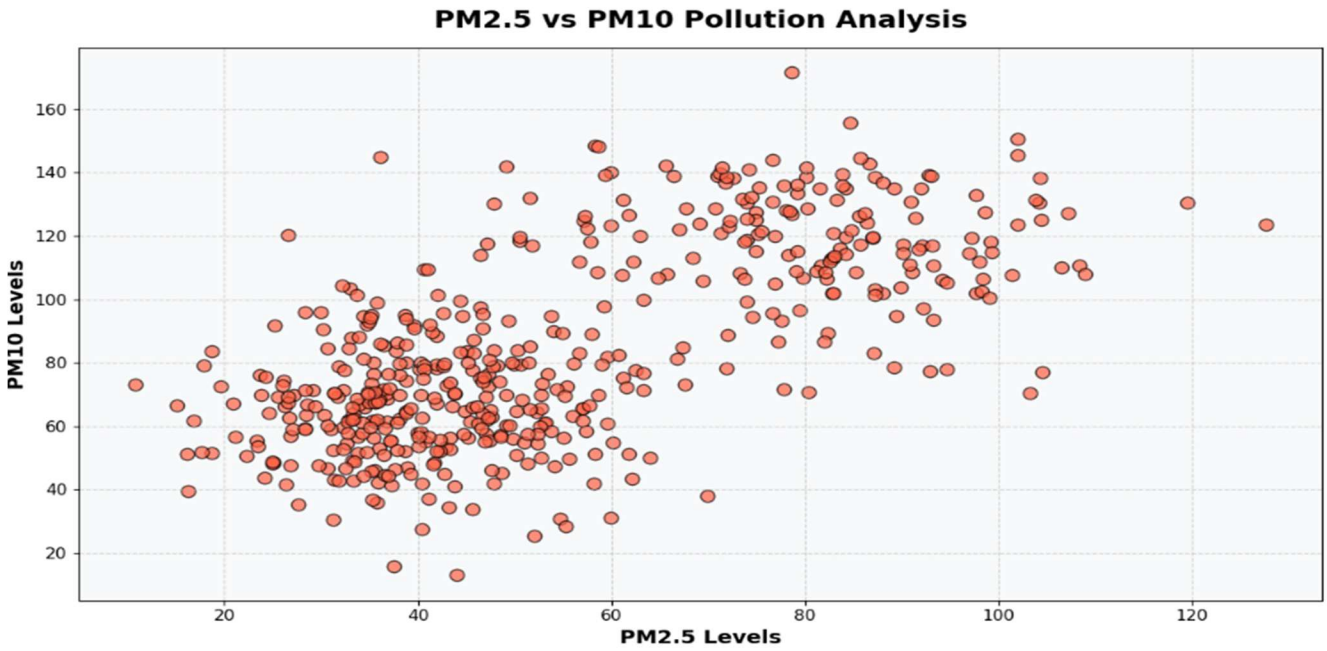
INSIGHT 4: PM2.5 VS PM10 POLLUTION ANALYSIS

Objective- **To analyze the relationship between PM2.5 and PM10 pollutants.**

Visualization Used-**Scatter Plot**

Code

```
plt.figure(figsize=(12,7))

# Customized Scatter Plot
plt.scatter(
    df_air['PM2.5'],
    df_air['PM10'],
    color='tomato',
    edgecolors='black',
    s=80,
    alpha=0.7
)

# Title
plt.title(
    'PM2.5 vs PM10 Pollution Analysis',
    fontsize=18,
    fontweight='bold',
    pad=15
)

# Axis Labels
plt.xlabel(
    'PM2.5 Levels',
    fontsize=14,
    fontweight='bold'
)

plt.ylabel(
    'PM10 Levels',
    fontsize=14,
    fontweight='bold'
)

# Grid
plt.grid(
    linestyle='--',
    alpha=0.5
)

# Background Styling
plt.gca().set_facecolor('#f8f9fa')

# Tick Styling
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

# Layout
plt.tight_layout()

plt.show()
```



Analysis

- PM2.5 and PM10 show a positive relationship.
- Higher PM2.5 levels usually correspond to increased PM10 concentration.
- Indicates deteriorating air quality conditions.

Smart City Insight

This helps authorities:

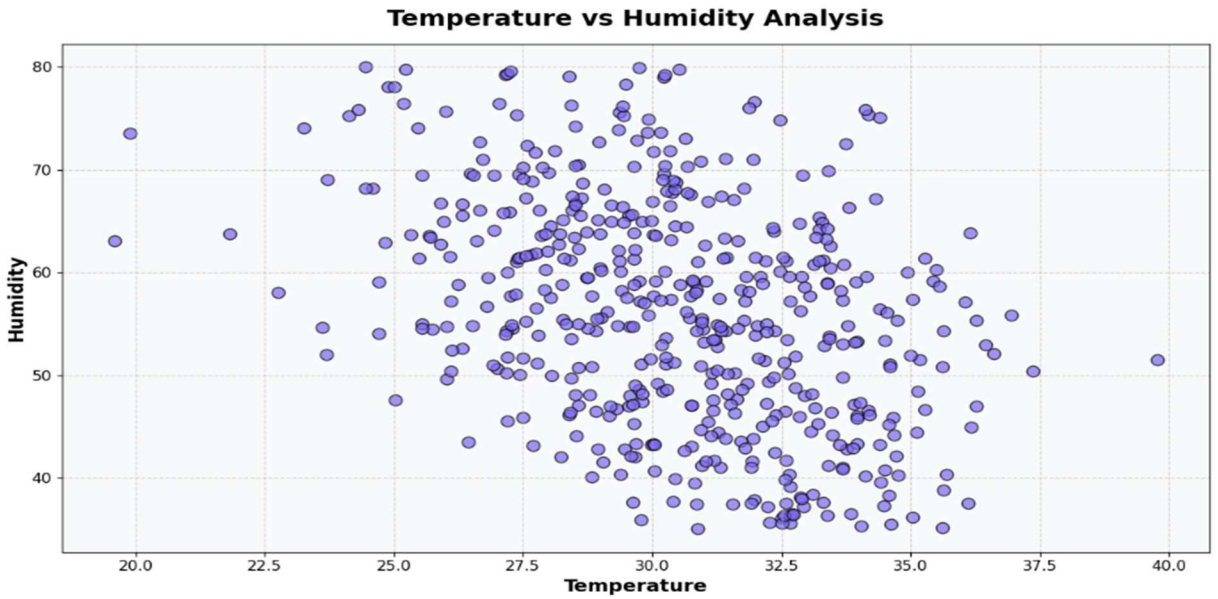
- monitor pollution levels
- identify hazardous areas
- improve environmental policies
- implement pollution control measures

INSIGHT 5: TEMPERATURE VS HUMIDITY ANALYSIS

OBJECTIVE: TO STUDY ENVIRONMENTAL WEATHER CONDITIONS.

- Visualization Used- Scatter Plot

```
plt.figure(figsize=(12,7))

# Customized Scatter Plot
plt.scatter(
    df_air['Temperature'],
    df_air['Humidity'],
    color='mediumslateblue',
    edgecolors='black',
    s=80,
    alpha=0.7
)

# Title
plt.title(
    'Temperature vs Humidity Analysis',
    fontsize=18,
    fontweight='bold',
    pad=15
)

# Axis Labels
plt.xlabel(
    'Temperature',
    fontsize=14,
    fontweight='bold'
)

plt.ylabel(
    'Humidity',
    fontsize=14,
    fontweight='bold'
)

# Grid
plt.grid(
    linestyle='--',
    alpha=0.5
)

# Background Color
plt.gca().set_facecolor('#f8f9fa')

# Tick Styling
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

# Layout
plt.tight_layout()

plt.show()
```



Analysis

- Temperature and humidity vary across observations.
- Some regions show higher humidity at lower temperatures.
- Weather conditions influence air quality behavior.

Smart City Insight

This analysis supports:

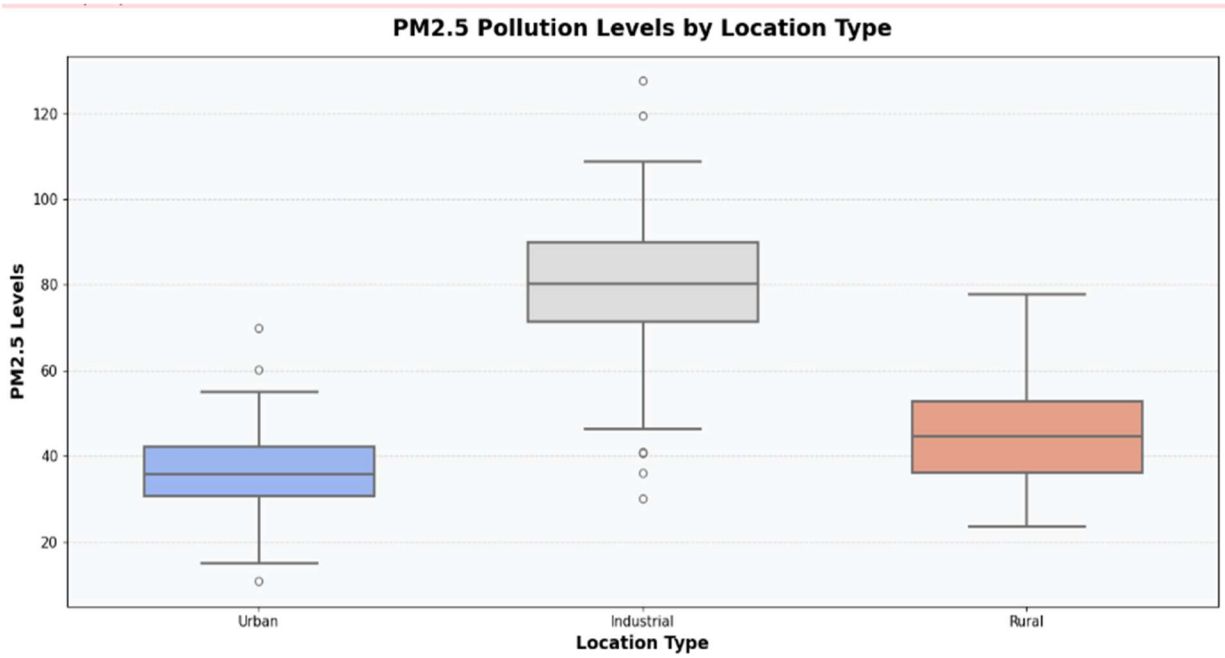
- climate monitoring
- environmental forecasting
- pollution prediction
- smart environmental management

INSIGHT 6: POLLUTION BY LOCATION TYPE

Objective- **To compare pollution levels across different locations.**

- Visualization Used-**Box Plot**

```
plt.figure(figsize=(14,7))

# Customized Boxplot
sns.boxplot(
    x='Location_Type',
    y='PM2.5',
    data=df_air,
    palette='coolwarm',
    linewidth=2,
    width=0.6
)

# Title
plt.title(
    'PM2.5 Pollution Levels by Location Type',
    fontsize=18,
    fontweight='bold',
    pad=15
)

# Axis Labels
plt.xlabel(
    'Location Type',
    fontsize=14,
    fontweight='bold'
)

plt.ylabel(
    'PM2.5 Levels',
    fontsize=14,
    fontweight='bold'
)

# Grid
plt.grid(
    axis='y',
    linestyle='--',
    alpha=0.5
)

# Background Styling
plt.gca().set_facecolor('#f8f9fa')

# Tick Styling
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

# Layout
plt.tight_layout()

plt.show()
```



Analysis

- Urban and industrial areas show higher pollution levels.
- Residential locations generally have lower PM2.5 values.
- Pollution variation differs across location categories.

Smart City Insight

This helps city planners:

- identify polluted zones
- improve environmental safety
- plan green zones
- support sustainable urban development

INSIGHT 7: TRAFFIC AND POLLUTION RELATIONSHIP

Objective- **To understand how increasing traffic affects pollution levels.**

- Visualization Used-**Comparative Analysis**

Analysis

- Areas with high vehicle movement tend to show higher pollution concentrations.
- Traffic congestion contributes to poor air quality.
- Increased emissions directly affect environmental health.

Smart City Insight

This insight helps:

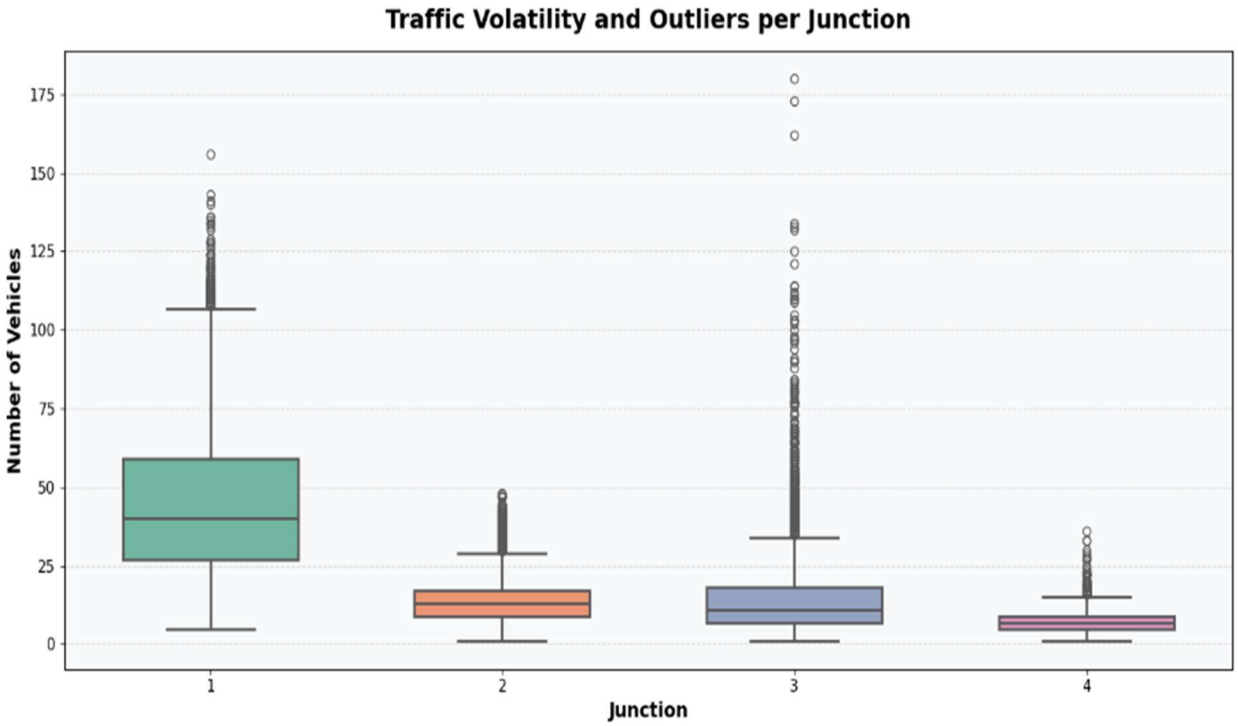
- reduce vehicular pollution
- encourage smart transportation systems
- improve public transport planning
- support eco-friendly city initiatives

Insight 8: Junction-wise Vehicle Volatility

Objective- To analyze traffic unpredictability and sudden vehicle surges.

Visualization Used – box plot

```
plt.figure(figsize=(14,7))

# Customized Boxplot
sns.boxplot(
    x='Junction',
    y='Vehicles',
    data=df_traf,
    palette='Set2',
    linewidth=2,
    width=0.6
)

# Title
plt.title(
    'Traffic Volatility and Outliers per Junction',
    fontsize=18,
    fontweight='bold',
    pad=15
)

# Axis Labels
plt.xlabel(
    'Junction',
    fontsize=14,
    fontweight='bold'
)

plt.ylabel(
    'Number of Vehicles',
    fontsize=14,
    fontweight='bold'
)

# Grid
plt.grid(
    axis='y',
    linestyle='--',
    alpha=0.5
)

# Background Color
plt.gca().set_facecolor('#f8f9fa')

# Tick Styling
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

# Layout
plt.tight_layout()

plt.show()
```



Analysis

- Junctions with many outliers experience sudden traffic spikes.
- High variability indicates unstable traffic conditions.
- Certain junctions are more congestion-prone.

Smart City Insight

This helps authorities:

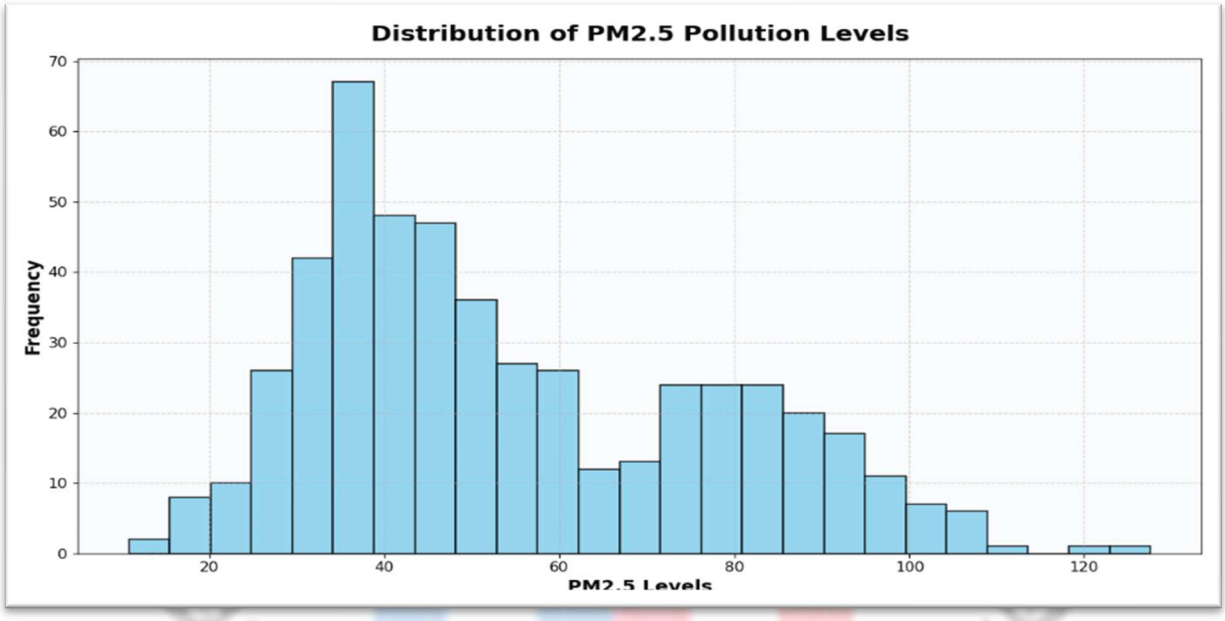
- implement Adaptive Traffic Control Systems (ATCS)
- manage unexpected traffic surges
- optimize signal timings dynamically
- improve smart traffic monitoring systems

INSIGHT 9: AIR POLLUTION DISTRIBUTION ANALYSIS

Objective- To understand the spread and concentration of PM2.5 pollution levels.

Visualization Used- histogram

```
plt.figure(figsize=(12,7))

# Histogram
plt.hist(
    df_air['PM2.5'],
    bins=25,
    color='skyblue',
    edgecolor='black',
    linewidth=1.2,
    alpha=0.9
)

# Title
plt.title(
    'Distribution of PM2.5 Pollution Levels',
    fontsize=18,
    fontweight='bold',
    pad=15
)

# Axis Labels
plt.xlabel(
    'PM2.5 Levels',
    fontsize=14,
    fontweight='bold'
)

plt.ylabel(
    'Frequency',
    fontsize=14,
    fontweight='bold'
)

# Grid
plt.grid(
    linestyle='--',
    alpha=0.5
)

# Background Color
plt.gca().set_facecolor('#f8f9fa')

# Tick Styling
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

# Layout
plt.tight_layout()

plt.show()
```



Analysis

- Most pollution values are concentrated within a moderate range.
- Extremely high PM2.5 values indicate hazardous pollution conditions.
- Distribution analysis helps identify abnormal environmental situations.

Smart City Insight

This helps in:

- identifying high-risk pollution zones
- improving environmental monitoring
- issuing public health alerts
- supporting pollution control policies

Insight 10 :Correlation Heatmap Analysis

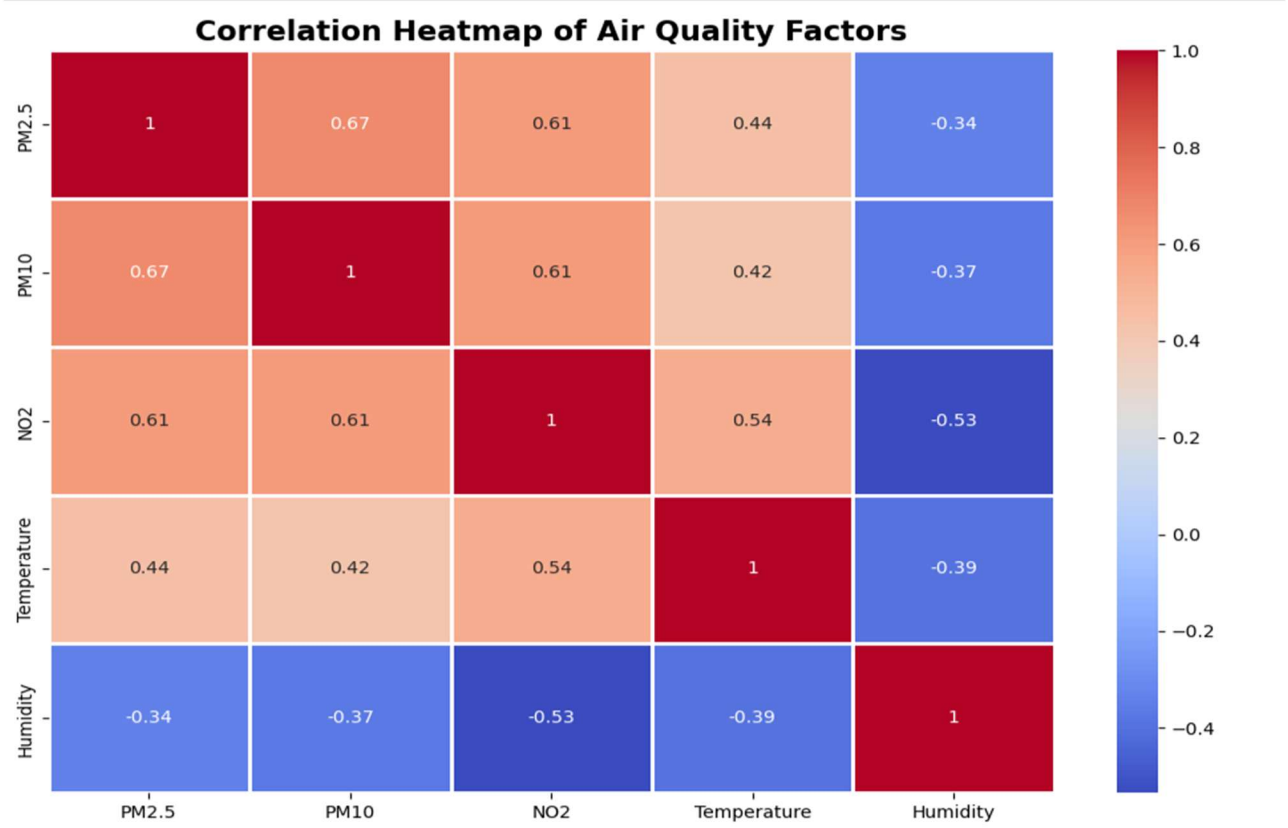
Objective

To identify relationships between air quality parameters and environmental factors using correlation analysis.

VISUALIZATION USED

- Heatmap (Correlation Matrix)

```
plt.figure(figsize=(10,7))

# Correlation Matrix
corr_matrix = df_air[
    ['PM2.5', 'PM10', 'NO2',
     'Temperature', 'Humidity']
].corr()

# Heatmap
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    linewidths=1
)

# Title
plt.title(
    'Correlation Heatmap of Air Quality Factors',
    fontsize=16,
    fontweight='bold'
)

plt.tight_layout()

plt.show()
```



SMART CITY INSIGHT

The correlation heatmap helps identify how different environmental factors and pollutants are related to each other.

- PM2.5 and PM10 show strong positive correlation, indicating that both pollutants increase together during poor air quality conditions.
- Temperature and humidity influence pollution patterns and environmental behavior.
- Correlation analysis helps identify major pollution indicators affecting urban environments.
- Strong relationships between variables support predictive environmental monitoring.

This insight can help smart city authorities:

- monitor pollution trends

- improve environmental policies
- develop pollution prediction systems
- support sustainable urban planning
- enhance public health monitoring systems

FUTURE SCOPE

The current analysis provides a baseline for urban health. Future iterations could include:

• Predictive Modeling: Using LSTMs (Long Short-Term Memory networks) to predict traffic 2 hours in advance.

• IoT Integration: Connecting real-time sensor feeds for a live dashboard.

• Health Impact Mapping: Correlating respiratory hospital admissions with the air quality spikes identified here.

Conclusion

The Smart City Analytics Dashboard successfully demonstrates how data visualization techniques can be used to analyze urban traffic and environmental conditions effectively. Using traffic and air quality datasets, multiple analytical insights were generated through various visualization methods such as line plots, bar charts, histograms, scatter plots, heatmaps, and box plots.

The project helped identify important smart city patterns including traffic congestion, pollution trends, junction-wise vehicle density, environmental variations, and pollution distribution across different locations. These insights can support better urban planning, traffic management, and environmental monitoring.

Python libraries such as Pandas, Matplotlib, and Seaborn provided efficient tools for data preprocessing and visualization. The use of graphical analysis made complex city data easier to understand and interpret.Overall, this project highlights the importance of data analytics and visualization in developing smarter, safer, and more sustainable cities.

REFERENCES

1. Open Source Datasets: Air Quality and Traffic Prediction datasets sourced for urban analytics.
2. Kaggle — Source for traffic and environmental datasets.
3. McKinney, W. (2017). *Python for Data Analysis*. O'Reilly Media.
4. Matplotlib Documentation — Official documentation for visualization techniques.
5. Seaborn Documentation — Statistical data visualization library documentation.
6. Pandas Documentation — Documentation for data analysis and manipulation.
7. VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media.
8. Python Official Documentation — Reference for Python programming concepts.
9. Batty, M. (2013). *The New Science of Cities*. MIT Press.
10. Smart City concepts and urban analytics research papers referenced for understanding traffic and environmental monitoring systems